Software Development Processes from the Developers' Perspective

Name 1
ID 1
email 1

Name 2
ID 2
email 2

Name 3
ID 3
email 3

ABSTRACT

The focus of this paper is to report the software development approaches described by A, B, and C during our interviews with them.

1. INTRODUCTION

Software development process consists of a series of technical, collaborative, and managerial activities, which aim to specify, design, implement, and test a software system [1]. It is with the careful selection of software processes that developing a great piece of software becomes achievable. This paper presents two different development processes which were described by two software engineers who work in two different companies, with two different end users. The aim of this paper is to identify similarities and differences between the development processes, identify their strengths and weaknesses, and suggest how each of these processes could be improved.

1.1 A

AA has graduated from McGill University in 2016, with a bachelor's degree in Software Engineering. He has been involved with internship opportunities in A, B, and C and is now working as a Software Engineer at D as a member of the XXX team [2].

1.2 B

B has also graduated from McGill University, but in 2017, with a bachelor's degree in Software Engineering. B has been previously hired as an intern at Microsoft. She has been working for C since 2017 as an intern, but started working there as a full time engineer since January of 2018, where she functions as a full stack developer under the d team for means of delivering a media processing software [3].

1.3 C

C has graduated from McGill University with a degree in Computer Engineering, and is employed at D Canada Inc. Although he has recently started working there as a full time software developer, he has previously been interning in this company for 2 years, prior to his full time employment [4].

1.4 More Information

This paper will consider A, B, and C description of the software development process at E, F, and G respectively.

2. COMPARING APPROACHES

2.1 Software Development Process

Regardless of the different types of software development processes used, which are to be elaborated on herein, all three of the development processes depend heavily on the feedback provided by the project managers.

2.2 Collaboration and Development Environment

As pointed out by the interviewees, all developers work in an environment where close ties between the software engineers and product managers exist, to ensure the designed software meets all the requirements criteria and no overlapping occurs between the assigned tasks among the software engineers.

2.3 Standups

All interviewees benefit from having a technical leader who functions as a scrum master, by ensuring that the standups happen correctly, covering the parking lot items and dealing with the relevant questions with regards to the project.

2.4 Quality Assurance

In all software development processes, quality assurance is one of the most important development phases that marks the end feature development in both waterfall and agile development processes. This development stage is especially important at the organizational level, as it ensures the development of a high quality product which adheres to the previously defined standards [1][5].

2.5 Team Coordination

For all companies, the team is coordinated by the engineers themselves through updating each other on the work they have done and the amount of progress they have made with regards to their tasks.

2.6 Distribution of Tasks

Regardless of the differences in the distribution of tasks at each of the companies, which has been described in the next section, all interviewees mentioned how a lot of effort is spent in the beginning to ensure no dependencies exist between the tasks. Moreover, the tasks are defined such that the development process does not heavily rely on the knowledge of only one of the team members.

2.7 Communication Overhead and Responsiveness

As mentioned by all the interviewees, communication among the team members is facilitated by the scrum master to minimize the amount of communication overhead, which is especially important for an agile development environment [6].

3. CONTRASTING APPROACHES

3.1 Software Development Process

According to the interviewees, A follows a waterfall approach, whereas B and C follow an agile development process. This difference is mainly due to different companies working on different products, even though they have same types of end users, which are companies as opposed to individuals. As indicated by E, the specifications that they need to follow for developing software are very concise and cogent, which need to be very top-down and decided for from the beginning [2]. Consequently, an agile development process would not be very applicable as the specifications cannot change much and specific chunks of the software need to be delivered on schedule. In contrast, at F, the software specifications are subject to change based on the received feedbacks from the project managers. Additionally, as mentioned by G, some of the user stories are even prone to get discarded altogether if major UI changes or failures occur during the development process [3]. H follows a similar style.

3.2 Requirements Process

By inspecting the transcripts from each of the interviewees, it can be realized that the main distinction between the implemented software development processes at each company is due to the project requirements. At A, the development team has to stick with the requirements which are predefined by product owners and product managers since the beginning. However, at B and C, the requirements are changeable by the project managers and product owners and some of the developed parts may get discarded due to a changes in the product backlog.

3.3 Division of Tasks

Except for A where tasks are assigned to developers by the scrum master, the team members get to choose what they would like to work on at the other two companies for most of the times. The task assignation process is purely based on the urgency of a feature to be delivered as well its specificity.

4. STRENGTHS

4.1 A Approach

4.1.1 Scrum Meeting

As mentioned by B, stand-up meetings take place on daily basis at A for means of status updates [2]; time-boxed daily scrum meetings can be very beneficial while developing software as it would enable software engineers to update each other on the tasks they are working on and if they have blocking issues. In that case, other members of the team may propose potential solutions or change the priority of the tasks to overcome the obstacles.

4.1.2 Distribution of Tasks

At B, software developers get to choose the task they would like to work on unless a very specific technique is required to accomplish something for which they would call out individuals for it. Allowing engineers to take on a task that they would like to be working on enables them to contribute to the project as much as they could. This would not only increase the engineer's productivity, it would also increase the development process' throughput.

4.1.3 Requirements Process

As previously mentioned, the requirements, for which a lot of time has been invested on in the beginning of the project, remain unchanged throughout the waterfall development process implemented at B. Having unchanging requirements can be beneficial in terms of maximizing the efficiency of the development process by excluding the need to start anew upon changing the requirements.

4.2 C Approach

4.2.1 Scrum Meeting

As stated in the interview, D team have a scrum/stand-off meetings every other days [3]. This is beneficial for a senior team which have been working together. As a developer can be in many other team, having multiple scrum/stand-off meeting could lead to them losing valuable time to work on each tasks. This also gives the developer the chance to come in with more detailed comments on their work and level of completions. This makes the meeting a more comprehensive one than an unproductive one, especially at the beginning of a new project.

4.2.2 Distribution of Tasks

As stated in the interview, the tasks are distributed by the scrum master with the project owner and manager. In situations where the developers cannot make it to the meeting or are new to the team, the scrum masters would have a better understanding of the tasks. This could help with new developers to get familiar with the new team structure and give them the time to learn about the team's time frame structure. This also gives the project owner and manager the time to also learn about the developers' strength and weaknesses. Also, if a senior member of the development team is absent during the grooming session, the project leaders can assign the appropriate task without compromising the work-flow due to their absence as they can be debriefed privately or during the stand-up/scrum meetings.

4.3 Z Approach

4.3.1 Scrum Meeting

The development team in C has a scrum meeting every day, at a fixed time early on during the day. This helps the team stay up-to-date with the progress of the current tasks in progress by the team members, as well as an overview of how close the team is to finishing the committed stories. Moreover, since they are conducted in the beginning of the day, all the team members get a clearer idea of what the focus of the day should be, and what the priorities of the team are, instead of spending time and resources on a task which may not be very prioritized in terms of team goals. Also, the scrum environment seem to be very

relaxed and friendly, where there is no specific progress report and analysis format required, but more of a discussion about encountered problems or solved issues, as well as an overall update.

4.3.2 Distribution of Tasks

There are always a set of tasks available on the tasks list (JIRA, OpenLM, etc.) for the developers to work on. The developers are free to take on any tasks they desire. The scrum master does not assign any tasks to the team members, and the members have full autonomy to discuss and organize between themselves who should be responsible for a given task. Often, the members with more experience in certain aspects will take on tasks that they are more familiar with, in order to save time and resources. However, any other member is at liberty to take on tasks that they are not comfortable with as a learning opportunity, or because none of the members are familiar with the task at hand that needs to be done. As previously mentioned, the tasks are not assigned by the scrum master, however, the scrum master would often point out which tasks are prioritized, as well as recommending certain developers to take on certain tasks, as they would see fit. But the final decision is up to the developer. As mentioned in the interview, Roman points out that they had worked on projects from the very beginning, and that how they would have total autonomy over what they should prioritize the work on, as well as distributing the responsibilities and the work to be done among themselves, without much intervention from the scrum masters or the project managers.

4.3.3 Story Creation and Story Points

Another one of the strengths in C is that during the sprint planning, as mentioned by E, the developers each sit down and evaluate the stories to be done individually, based on a point system. The more points the story has, the more time and resources need to be spent on that story. The team members each decide on a certain amount of story points, then reveal their numbers all at the same time, as mentioned by E, on the count of three, so that the member do not, involuntarily or not, influence other members' decisions [4]. This way, the team will get a better idea of what the story is actually worth, since the entire team votes on it, rather than it being a single person's decision. This is really important, because the team, more than anyone else, is aware of their own capabilities and limitations. Hence, no unrealistic expectations or goals are set this way, and the team is less likely to be overworked and overstressed.

5. WEAKNESSES

5.1 F Approach

5.1.1 Requirements Process

After asking G to provide an example of a user story which resulted in the failure of a project within F, he mentioned how one of the user stories associated with the G project could not be fully implemented due to running into additional complexities which were not envisioned by the software architect from the beginning [2]. Implementing a waterfall process as the one exploited at F can have such downsides as it won't allow the software engineers to change the requirements to overcome blockers. Furthermore, such waterfall requirements processes leave no room for revision for such unplanned road-blockers as the waterfall process implemented at this company needs to follow a strict project schedule, as pointed out by the interviewee.

5.1.2 Distribution of Tasks

Allowing the team members to take on a task they enjoy the most can effectively increase their productivity, which would in turn increase the development process throughput. However, this can have its own downsides. If the same team member continues to work on similar tasks in the future, the development team is risking concentrating the knowledge or a certain technique on one person. Moreover, the developers may not be aware of the complexity of a certain task they would like to work on, and their insufficient knowledge for accomplishing it may lead to project failures when time is a constraint.

5.2 C Approach

5.2.1 Scrum Meeting

Unlike D, in C, the project she is part of, she would do scrum meeting every other day compared to everyday. The understanding behind this is that the developers should maximize their work efforts while having a short standoff every other day. As the project progresses, and as the product comes closer to the release period, a more consistent daily meeting can help with any problem and questions that may arise as messages and emails could be easily left unread as the developers are concentrating on the final details of the product; this could create a miscommunication, misunderstanding, and waste of time while trying to privately communicate to each members of the team. Moreover, this would slow down the communication between the remote members and the senior members of the team while remote members are trying to acquire important information and feedback.

5.2.2 Distribution of Tasks

One of the main weakness that can be noted is how tasks are distributed between developers at the start of a sprint. As G stated in the interview, the scrum master with the project owner and manager would choose the task for the developers. This practice can have consequences in a long term if the managers do not give opportunity for developers to widen their reach of the product. This could lead to lack of interest, lack of commitment, or limiting the developer from learning and developing their skills which can benefit the team and the company in the future. For the developers, this could give lower the product quality and life span as disinterest could lead to a member of the team leaving the project or doing the minimum work required as there is no personal incentive/commitment made by them for the tasks asked.

5.2.3 Story Creation and Story Points

As stated in the interview, representing a story point by a QA member seems very unusual. Just like the distribution of tasks, setting the story points by a non-developer and another member of the team is not something that an agile methodology would suggest. In the previous team that she had worked for, during the planning period, they would do planning poker to determine the points as a team. Adding one voice or a few voices only to associate a point to a story point could result in a backfire if not

processed properly as the developer working on the story could find him/herself in a pickle.

5.3 G Approach

5.3.1 Scrum Meeting

As mentioned in previous section, the members in the agile teams have a wide range of autonomy. This can cause problems under wrong circumstances, since there is always the possibility of members taking advantage of this, while barely making any real progress or contribution to the tasks at hand. This was not directly mentioned from the interview with H, but he mentioned being involved in several POC (proof of concept) projects [4]. These types of projects require a certain amount of dedication and work etiquette, and most importantly, since they are more of a research and prototyping kind of projects, the team members come up with their own tasks and requirements, since there are no fixed set of tasks. With this kind of Agile development approach, a member has the opportunity to not accomplish much, without the rest of the team, and the scrum master, being aware of it.

5.3.2 Distribution of Tasks

As previously mentioned, no real weakness is apparent when it comes to task distribution at H, since the tasks are picked up by the members of the Agile team themselves, rather than the tasks being assigned to them by the scrum master or the project owner. However, this type of approach can have its pitfalls too. For example, a team member might choose a task that they think is reasonable and within their grasp of knowledge, only to find out later on that it is actually not what they thought it to be, which can introduce difficulties, such as the possibility of re-assigning the task to another team member which itself can disturb the flow of the team and the workload between the members, or the possibility of the member trying to complete the task and being stuck on them for a long time, or even possibly the entirety of the sprint, which could cost the team a tremendous amount of time and resources, as the member's abilities could have been better utilized elsewhere, while another member could have completed the blocking task at hand. Moreover, as mentioned by Roman, there might be new tasks coming in on a weekly or even a daily basis, which might hinder the distribution of the workload, and the overall progress of the sprint.

5.3.3 Story Creation and Story Points

In the interview, X mentions that he works on a POC (proof of concept) project for now, and that involves understanding what needs to be done, and how to approach the architecture and development of this project [4]. Given the understanding that no one in the team has a strong understanding of what needs to be done, coming up with user stories can prove to be a bit difficult and inaccurate, since there is a lot of room for misunderstandings, or prioritizing unimportant tasks. Moreover, since the user story to be implemented would be first of its kind, the team's estimation for the said user story might be off by a considerable amount, even after using their group's user story points estimation system. These cases might end up causing the team to be not on time for their estimated delivery, and inadvertently wasting company's time and resources.

6. IMPROVEMENTS

6.1 M Approach

6.1.1 Development Process

One of the improvements that could be suggested for the development process at M would be switching to an agile development process. One of the main advantages of agile processes is their tolerance to a change in the requirements which facilitates delivery of a functional and valuable software [9]. Although M is executing a waterfall model for developing its products mainly due to time constraints, implementing an agile methodology for producing software can lead into a decrease in the time spent for dealing with blockers by allowing changes to be made to the user stories for means of making them simpler and removing unnecessary complexities. Moreover, upon adopting to an agile software development method, architectural changes can be easily minimized by exploiting a test-driven development; " incorporating appropriate tests for performance, scaling, and reliability—goes a long way to avoiding redundant, poorly fitting, and costly over-engineered solutions"[6].

6.1.2 Distribution of Tasks

At times where time is not a constraint, the technical leader/scrum master should assign team members to tasks which they have never been exposed to before, such that the developers gain new knowledge or skills. With this manner, if one of the team members were not able to contribute to the project for justified reasons, other engineers in the team would be able to successfully manage and finish their team member's tasks since they have the sufficient knowledge to do so.

6.2 M Approach

6.2.1 Scrum Meeting

For the scrum meeting/stand-up, as stated by N, there should just be enough meeting during the week to eliminate any overhead which could lower the productivity of members of the team. As such, the every-other-day approach is reasonable. However, having the ability to modify the number of meeting days as the project progresses could increase productivity as everyone would be on the same page. Communication is key for developers, project owner/manager, and scrum master be on the same level of understanding of the outcome of the project.

6.2.2 Distribution of Tasks

For the distribution of task, as part of an agile methodology, developers should have the ability to choose which story to work on. Scrum master should not be the one giving out the task as their job is to make sure that the work being done by the developer follows the story objectives and also to not give any personal feedback on the matter which could influence the work-flow of a developer. It is good to suggest or ask a member of the development team to take on a particular story if the team believes that he/she is more fit but never demand from them. Trust is the name of the game which is crucial for the well-being of the team.

6.2.3 Story Creation and Story Points

The main criticism for the agile method done by her team is how the story point is dictated by a QA other than the developer who would work on the story or by the team. This practice is something that is difficult to understand. Letting other members of the team, who are not a developer, dictate the time frame of the work could affect the productivity of the developers. As such, they should use the agile method of setting the point by applying the pointing poker to set the difficulty level of the project and then let the developers and the QA to add their own point of view to have a common point ground for each story. This will make sure that the story correctly reflects the estimated cost to develop and test at the same time. Setting point according to the time required to test the story only is not very productive and could potentially lead into the overlap of multiple stories between sprints.

6.3 O Approach

6.3.1 Scrum Meeting

From the comparison of O's strength and weaknesses regarding scrum, the underlying potential problem could be the amount of autonomy a team member has over what task they choose, what type of task they choose, and how they report it during the daily scrum meetings. Too much control over the team members can result in the under-performance of the team, since the members might be assigned to tasks that they are not good at, or are not very passionate about. However, at the same time, problems might arise from too much autonomy as well. A sensible way to improve this is to let the members enjoy the same autonomy when choosing tasks among themselves, but then having to report back with an expected amount of work done during the next meeting, or in the case of a blockage, the reasons of the blockage and the possible solutions attempted or to be attempted. This will ensure a proper use of team's time and resources, and consequently a healthy amount of progress even in a not desirable scenario, for example POC.

6.3.2 Distribution of Tasks

It was found that there may be some issues with the task distribution in W Agile teams, if the member chooses a task without completely knowing what the task consists of, which can be problematic for the team. In order to counter this problem, there should be a task breakdown available for the members to view, so that it can reduce the chances of a member fully committing to a priority task they are unsure of that needs to be accomplished. The ultimate goal is to reduce the cost of time and resource for the team and the company, and ensuring that there is little room for any misunderstandings and untimely commitments is integral to this process.

6.3.3 Story Creation and Story Points

The main issue with story creation and story points seemed to be when the team is tasked to work on a new project, or a POC (proof of concept). With these types of assignments, there will always be some sort of problems with the story creations and story point estimation, since the team does not have a strong enough grasp on the project to be able to concretely estimate the work to be done. However, there are ways to narrow down this inaccuracy to a minimum. An example of this would be to do enough research on the subject matter before starting any sort of design and implementation, in order to minimize the process of going back to the drawing board and coming up with new ways to implement things. Another way would be to find a person with enough experience on the subject matter, and temporarily lending him to the developing team, in order to smooth out the researching and development process.

7. ACKNOWLEDGEMENT

We like to thank all our interviewees A, B, and C for taking off of their valuable time to let us learn about the methodology used in their software development process and for providing us with examples regarding successful and unsuccessful user stories and functional/nonfunctional requirements. We would like to also thank D, E, and F for having learned how each of them utilize software development methods in different processes to accomplish tasks. From this, we were able to learn and understand how each project can be different from one another based on its requirements criteria. Furthermore, we were able to find strengths as well as weakness associated with each process, for which we suggested improvements for, which can be applied to the future projects that we will be part of.

8. REFERENCES

[1] I. Sommerville, Software engineering. Boston: Pearson, 2011.

[2] S001, 2018. Interview Transcript (A).

[3] S002, 2018. Interview Transcript (B).

[4] S003, 2018. Interview Transcript (C).

[5] Stamelos, I. and Sfetsos, P. 2007. Agile software development quality assurance. Information Science Reference, Hershey, PA.

[6] Ali Babar, M., Brown, A. and Mistrík, I. (n.d.). *Agile software architecture*.